

AI FOCUS — 26 QUESTIONS

ReportFlow AI Module Interview Guide

PDF/DOCX Pipeline · OCR · LLM · Celery · Privacy · Best Practices

26 interview questions with clear, detailed answers in simple English.

ReportFlow · Django College Report Management System
Covers architecture, groups, workflow, security, API, certificates, AI & more.

How to use this guide

Read each question like an interviewer might ask it. The answer explains **what** the feature is, **why** we built it that way, and **how** it works in ReportFlow. Practice saying answers in your own words — do not memorize word-for-word.

Sections in this PDF

- **1. AI Features Overview** — 3 questions
 - **2. PDF Extraction & OCR Pipeline** — 6 questions
 - **3. LLM Integration & Teacher Insights** — 5 questions
 - **4. Async Processing, Settings & Security** — 4 questions
 - **5. Interview Deep-Dive (AI Scenario Questions)** — 8 questions
-

1. AI Features Overview

Q1. What AI features does ReportFlow provide?

AI Report Analysis — extract PDF/DOCX text, run OCR on scans if needed, verify PDF quality, generate summary and rubric score suggestions for teachers.

AI Q&A; assistant — optional AI-powered help for user questions (AIReportService / AIQAService).

AI is assistive only — teachers and admins always make final decisions; AI does not auto-approve reports.

Controlled by AI_FEATURES_ENABLED setting and runs mainly on teacher request or background Celery task.

Q2. Why add AI to a report management system?

Teachers spend long hours reading lengthy PDFs. AI gives a quick summary and draft rubric scores to speed review.

OCR verification catches scanned/low-quality PDFs where text extraction fails.

Reduces repetitive work while keeping human judgment for marks and approval.

Demonstrates modern full-stack skills: LLM integration, PDF pipeline, async processing.

Q3. Where is AI code located in the project structure?

application/services/ai_report_service.py — main orchestration.

infrastructure/ai/document_extractor.py — routes PDF vs DOCX extraction.

infrastructure/ai/pdf_extractor.py — native PDF text via PyMuPDF.

infrastructure/ai/llm_client.py — OpenAI/HTTP LLM calls with JSON response.

infrastructure/ocr/pdf_ocr.py — Tesseract OCR fallback (PDF only).

tasks/celery_tasks/ai_tasks.py — process_report_ai_task for async runs.

2. PDF Extraction & OCR Pipeline

Q4. Explain the full AI report analysis pipeline step by step.

- 1) Load report file (PDF or DOCX) from disk. 2) Extract text — PyMuPDF for PDF, zip+xml for DOCX (document_extractor.py).
- 3) For PDF only: if native text is short, run OCR with pytesseract on page images.
- 4) verify_pdf_text() compares native vs OCR to detect scan quality issues (PDF only).
- 5) Send extracted text + rubric criteria to LLM (or heuristic fallback). 6) Save on Report JSON fields.

Q5. Does AI work on DOCX submissions?

Yes. document_extractor.py detects file type and reads word/document.xml from the DOCX zip.

OCR verification applies to PDF only — DOCX skips verify_pdf_text() but still gets LLM/heuristic analysis.

Teachers see the same AI insights panel on report detail for both PDF and DOCX reports.

If extraction fails, ai_processing_status=FAILED with a friendly error; manual review still works.

Q6. What is PyMuPDF used for in this project?

PyMuPDF (fitz) reads PDF files and extracts embedded text without OCR.

It is faster and more accurate than OCR when students submit proper digital PDFs.

Returns text, page count, and char count capped by AI_MAX_PDF_TEXT_CHARS setting.

First choice in the pipeline — OCR only when native extraction is insufficient.

Q7. When and why is OCR (Tesseract) used?

OCR runs when native text has fewer characters than AI_OCR_MIN_NATIVE_CHARS (default 400).

Common for scanned reports photographed or printed and uploaded as PDF.

pytesseract converts PDF pages to images then reads text.

Slower and less accurate than native text — but necessary for image-based PDFs.

Q8. What does verify_pdf_text() do?

Compares native extracted text with OCR text to assess PDF quality.

Stores results in report.ocr_verification_json: match score, warnings, page count, processed_at.

Helps teachers see if the PDF is searchable/digital or a low-quality scan.

Teachers can ask students to resubmit a proper PDF if verification fails badly.

Q9. What is stored in extracted_text, ai_analysis_json, and ocr_verification_json?

extracted_text — plain text from PDF (native or OCR) or DOCX used for AI and display.

ai_analysis_json — summary, suggested_criterion_scores, suggested_feedback, suggested_teacher_marks, provider, generated_at.

ocr_verification_json — quality metrics from verify_pdf_text().

ai_processing_status — PENDING, PROCESSING, COMPLETED, or FAILED.

3. LLM Integration & Teacher Insights

Q10. How does the LLM client work?

LLMClient in infrastructure/ai/llm_client.py wraps HTTP calls to OpenAI-compatible API via httpx.

chat_json(system_prompt, user_prompt) asks the model to return strict JSON.

If API key missing or call fails, heuristic_report_analysis() provides rule-based fallback.

Keeps AI optional — app works fully without any LLM configured.

Q11. What JSON does the LLM return for report analysis?

Expected keys: summary, suggested_criterion_scores (map of criterion id → score), suggested_feedback (text), suggested_teacher_marks (0–100 integer).

System prompt instructs model to stay concise and respect rubric max scores.

Response is parsed, validated, and normalized before saving to ai_analysis_json.

Q12. What is get_teacher_insights() and when is it called?

API/service method that returns AI summary, OCR verification, rubric suggestions for a report.

Only teachers and admins can access — permission check in AIReportService.

If status is PENDING, it triggers analyze_report_pdf() first then refresh_from_db().

Frontend report_ai.js displays insights panel on teacher report detail page.

Q13. Does AI auto-approve or auto-reject reports?

No. AI only suggests — teacher must manually enter marks, feedback, and click approve/reject.

This is intentional for academic integrity and legal accountability.

AI suggestions pre-fill forms optionally; final submission checkbox remains teacher's decision.

Good answer in interviews: "AI assists, humans decide."

Q14. How are AI failures handled gracefully?

On exception, ai_processing_status = FAILED and ai_analysis_json stores a safe error message.

Exception is logged to reportflow.ai logger. API returns user-friendly copy via user_messages / AppError.

Teacher can still review PDF/DOCX manually — AI failure never blocks approval workflow.

Retry possible by calling analyze again after fixing API key or file issue.

4. Async Processing, Settings & Security

Q15. How is AI processing run asynchronously?

`queue_report_ai_analysis(report_id)` in `tasks/dispatch.py` enqueues Celery task.

`process_report_ai_task` calls `AIReportService().analyze_report_pdf(report_id)`.

In development `CELERY_TASK_ALWAYS_EAGER=True` runs task immediately in same process.

Production uses Redis broker so long PDF+LLM work does not timeout HTTP request.

Q16. What settings control AI behavior?

AI_FEATURES_ENABLED — master switch to disable all AI endpoints.

AI_MAX_PDF_TEXT_CHARS — limits text sent to LLM (cost + token control).

AI_OCR_MIN_NATIVE_CHARS — threshold to trigger OCR fallback.

OpenAI API key and model name from environment variables — never hard-coded in repo.

Q17. What data is sent to the external LLM and what privacy concerns exist?

Sent: report title, rubric criteria names/max scores, extracted report text from PDF or DOCX (truncated).

Not sent: passwords, unrelated student records, admin settings.

For production colleges: need student consent, data processing agreement, possibly anonymize names.

Can disable AI entirely with `AI_FEATURES_ENABLED=False` for strict privacy policies.

Q18. Explain AIQAService / AI-assisted Q&A; if asked.

Separate from report analysis — helps answer user or visitor questions using LLM + FAQ context.

Uses `ai_qa_service.py` and api serializers; may suggest draft admin replies.

Keeps human admin in the loop for published answers — similar assist-only pattern.

Shows extensibility of LLMClient beyond PDF grading.

5. Interview Deep-Dive (AI Scenario Questions)

Q19. How would you improve the AI module in version 2?

Add chunking for very long PDFs with map-reduce summarization.

Cache analysis results by file hash to avoid re-processing unchanged resubmissions.

Support local/on-prem LLM (Ollama) for colleges that ban cloud AI.

Show confidence scores and highlight PDF sections used for each rubric suggestion.

Q20. How do you test AI features without spending API money?

Use `heuristic_report_analysis()` fallback when `OPENAI_API_KEY` is unset.

`CELERY_TASK_ALWAYS_EAGER=True` for synchronous tests in development.

Mock `LLMClient` in unit tests to return fixed JSON.

Test OCR path with sample scanned PDF fixtures in `media/test/`.

Q21. What is the difference between native PDF text and OCR text?

Native text comes from PDF's embedded character map — exact, selectable, fast.

OCR guesses text from pixels — errors on handwriting, skew, low resolution.

ReportFlow prefers native; OCR is fallback and quality-check reference.

Interview tip: mention you understand both digital-first and scan-first student submissions.

Q22. Why cap extracted text at `AI_MAX_PDF_TEXT_CHARS`?

LLM APIs charge by tokens — long theses could be expensive.

Model context windows have limits — truncating prevents API errors.

First N characters usually contain abstract, introduction, conclusion cues for summary.

Setting is configurable per deployment size (small college vs research university).

Q23. Describe `ai_processing_status` state machine.

PENDING — never analyzed yet. **PROCESSING** — job running now.

COMPLETED — success, JSON fields populated. **FAILED** — error stored, manual review still OK.

Status shown in teacher UI so they know if suggestions are fresh or unavailable.

Simple enum on Report model — easy to query and display.

Q24. How would you explain `AIReportService` in one paragraph to a non-technical interviewer?

"When a teacher opens AI insights, the system reads the student's PDF or Word file, checks if the text is clear (PDF scans), and sends a shortened version to an AI model along with the grading rubric. The AI returns a plain-English summary and suggested scores. The teacher reads everything

and decides the real marks — the AI is like a smart assistant, not the final judge."

Q25. What libraries would you mention for AI/PDF in this project?

PyMuPDF — PDF text extraction. **pytesseract** — OCR (PDF scans). **Pillow** — images for QR, OCR, certificate analyzer.

httpx — HTTP to LLM API. **Celery** — background analysis and certificate emails.

ReportLab + qrcode — certificate PDF generation (separate from AI but often asked together).

DOCX text uses Python zipfile + XML parsing — no python-docx dependency required.

Q26. Can the app run without installing Tesseract or OpenAI?

Yes. Core report workflow (submit, approve, certificate) needs neither.

Without Tesseract: OCR path fails gracefully; digital PDFs still work via PyMuPDF.

Without OpenAI: `heuristic_report_analysis()` gives basic keyword-based suggestions.

AI is an enhancement layer — good architecture keeps core features independent.